Bachelor Thesis

Computer Science and Engineering 300 credits

ASCON Acceleration on Xilinx Alveo U280 for V2X-Oriented Systems: Throughput Bottleneck Analysis and Energy-Efficient FPGA Design

Computer Science and Engineering 15 credits

Halmstad June 7, 2026

Seid Hackovic

HALMSTAD UNIVERSITY

Abstract

Vehicle-to-Everything communication systems require cryptographic processing of a large number of safety-critical messages under timing constraints. Earlier work has identified this as a potential bottleneck in dense traffic cases. This thesis examines whether the ASCON-128 authenticated encryption algorithm, standardized by the National Institute of Standards and Technology in 2023, can meet the requirements of such workloads when implemented as a hardware accelerator on a field-programmable gate array. A hardware core supporting both encryption and decryption was implemented in Verilog, using a round-based, iterative finite-state machine architecture, and was verified for correctness against official Known Answer Test vectors. Performance was evaluated through simulation at 100 megahertz on two platforms, an Artix-7 and an Alveo U280, and power was estimated using post-implementation analysis with Switching Activity Interchange Format (SAIF). The implementation achieved a message rate between 362,319 and 1,923,077 messages per second, depending on message size, exceeding the 4000 messages per second target by a factor of 90 to 423. Peak throughput reached 1159 megabits per second and all messages were processed within 2.76 microseconds. Logic power was estimated at 7 milliwatts on both platforms, and energy per message ranged from 0.102 to 0.477 microjoules on the Artix-7. The results demonstrate that a single ASCON-128 core at 100 megahertz meets the evaluated performance and energy requirements, supporting the use of ASCON-128 as a lightweight cryptographic building block in Vehicle-to-Everything oriented field-programmable gate array systems.

Keywords ASCON-128, authenticated encryption, Vehicle-to-Everything, field-programmable gate array, hardware accelerator, lightweight cryptography, Verilog

Sammanfattning

Vehicle-to-Everything Kommunikationssystem kräver kryptografisk bearbetning av ett stort antal säkerhetskritiska meddelanden under tidsbegränsningar. Tidigare studier har identifierat detta som en potentiell flaskhals i fall med tät trafik. Denna avhandling undersöker huruvida den autentiserade krypteringsalgoritmen ASCON-128, standardiserad av National Institute of Standards and Technology år 2023, kan uppfylla kraven för sådana arbetsbelastningar när den implementeras som en hårdvaruaccelerator på en fältprogrammerbar grindmatris. En ASCON-128 hårdvarukärna som stöder både kryptering och dekryptering implementerades i Verilog, med en rundbaserad iterativ finit tillståndsmaskinarkitektur och verifierades mot officiella Known Answer Test-vektorer. Prestandan utvärderades genom simulering vid 100 megahertz på två plattformar, Artix-7 och en Alveo U280, och effekten uppskattades med hjälp av post-implementeringsanalys med fullständig annotering av switchaktivitet. Implementeringen uppnådde en meddelandehastighet på mellan 362 319 och 1 923 077 meddelanden per sekund beroende på meddelandestorlek, vilket översteg målet på 4 000 meddelanden per sekund med en faktor 90 till 423. Den maximala genomströmningen uppmättes till 1 159 megabit per sekund och alla meddelanden bearbetades inom 2.76 mikrosekunder. Logikeffekten uppskattades till 7 milliwatt på båda plattformarna, och energin per meddelande varierade från 0.102 till 0.477 mikrojoule på Artix-7. Resultaten visar att en enda ASCON-128-kärna vid 100 megahertz uppfyller de utvärderade prestanda- och energikraven, vilket stöder användningen av ASCON-128 som en lättviktskryptografisk byggsten i Vehicle-to-Everything-orienterade fältprogrammerbara grindmatrissystem.

Nyckelord ASCON-128, autentiserad kryptering, fordon-till-allt, fältprogrammerbar grindmatris, hårdvaruaccelerator, lättviktig kryptografi, Verilog.

Preface

I would like to sincerely thank Hazem Ali for his guidance, support, and valuable feedback throughout this thesis. His insights and constructive advice played an important role in shaping both the direction and quality of this work. I am truly grateful for his encouragement and for the opportunity to develop my skills during this project.

Contents

Abstract . . . . . **iii**
Sammanfattning . . . . . **v**
Preface . . . . . **vii**
Contents . . . . . **ix**
List of Figures . . . . . **xi**
List of Tables . . . . . **xiii**
1 Introduction **1**
1.1 Background 1
1.2 Problem Statement 2
1.3 Purpose and goals 3
1.4 Limitations 4
1.5 Related Work 4
2 Technical Background **7**
2.1 Lightweight Cryptography 7
2.2 Overview of ASCON 7
2.3 ASCON-128 AEAD Operation 8
2.4 Structure of the ASCON 9
2.4.1 Round Constant Addition 9
2.4.2 Substitution Layer 9
2.4.3 Linear Diffusion Layer 9
2.5 Hardware Evaluation Considerations 10
2.6 V2X Workload Characteristics 11
3 Method **13**
3.1 Workload and Performance Targets 13
3.2 Development Process 13
3.3 Technical Design 14
3.4 Evaluation Methodology 14
3.4.1 Performance metrics 16
3.4.2 Power Estimation 16
3.5 Reproducibility 17
4 Implementation **19**
4.1 Module Structure 19
4.2 `Ascon_round Submodule` 19
4.3 Helper module (`ascon_helpers.vh`) 21

4.4 Finite State Machine . . . 21
4.5 Round Constant Generation . . . 23
4.6 Datapath and Block Processing . . . 23
4.7 Endianness and Key Handling . . . 24
4.8 Design Rationale . . . 24
5 Results . . . 27
5.1 Functional Correctness Verification . . . 27
5.2 Resource Utilization . . . 27
5.3 Latency and Throughput . . . 28
5.4 Power Estimation . . . 28
5.5 Energy per Message . . . 30
6 Discussion . . . 33
6.1 Performance Analysis . . . 33
6.2 Throughput Behavior and Architecture Trade-offs . . . 34
6.3 Resource Utilization . . . 34
6.4 Power and Energy Efficiency . . . 35
6.5 Validity of Results . . . 35
6.6 Development Process . . . 36
6.7 Societal Aspects . . . 37
7 Conclusion . . . 39
7.1 Future Work . . . 40
Bibliography . . . 41
A Appendix . . . 43

List of Figures

2.1 AEAD Structure Encrypt [12] 8
2.2 AEAD Structure Decrypt [12] 8
2.3 Illustration of vertical bit slicing in ASCON. Each highlighted column consists of one bit from each of the five 64-bit state words (S_0–S_4), forming a 5-bit input to the S-box [12] 9
2.4 Bit-sliced 5-bit S-box applied to the columns of the five 64-bit state words [12] 10
2.5 Linear diffusion layer: Each 64-bit word is XORed with two rotated versions of itself [12] 10

3.1 Simplified overview of the iterative ASCON round execution flow . 15

4.1 Architecture overview of the ASCON core, showing the FSM, state registers, permutation round, and datapath components. 20
4.2 Control flow of the ASCON encryption process, including initialization, associated data processing, plaintext encryption, and finalization. 22

5.1 Latency as a function of message size at 100 MHz. 29
5.2 Throughput as a function of message size at 100 MHz. 29
5.3 Energy per message for Artix-7 and Alveo U280 across evaluated message sizes. 31

A.1 Post-implementation power report for the ASCON design on the Alveo U280 platform. 43
A.2 Post-implementation power report for the ASCON design on the Artix-7 platform. 44
A.3 Post-synthesis power report for the ASCON design on the Alveo U280 platform. 44
A.4 Post-synthesis power report for the ASCON design on the Artix-7 platform. 45
A.5 Throughput evaluation results for the ASCON implementation on the Alveo U280 platform. 45
A.6 Throughput evaluation results for the ASCON implementation on the Artix-7 platform. 46

List of Tables

4.1 ASCON round constants for p^{12} and p^8 permutations. 24

5.1 Post-implementation resource utilization on Artix-7 and Alveo U280. 28
5.2 Measured performance at 100 MHz. Cycle counts are platform-independent. 28
5.3 Post-implementation power estimation on Artix-7 and Alveo U280. 30
5.4 Energy per message for Artix-7 and Alveo U280, calculated as total on-chip power × latency. 30
5.5 Requirement Verification Summary 31

6.1 Comparison of ASCON-128 implementations on Artix-7 FPGA . . . 34

Chapter 1

Introduction

Current communication systems rely on secure and efficient processing of large volumes of data under strict timing constraints. This is notably evident in emerging application domains such as Vehicle-to-Everything (V2X) communication, where safety-critical messages must be processed in real time under high traffic conditions.

In this context, cryptographic systems are necessary to ensure data authenticity and integrity for messages transmitted. However, the computational challenge of processing a large number of short messages under a tight latency constraint is significant. As a result, cryptographic processing can become a system-level bottleneck, limiting scalability and performance.

Hardware acceleration using field-programmable gate arrays (FPGAs) enables high-throughput, energy-efficient execution of cryptographic operations. In particular, server-type FPGA platforms such as the Xilinx Alveo U280 enable the offloading of computationally intensive workloads, making them suitable for high message-rate scenarios, such as V2X.

This thesis investigates whether the lightweight authenticated encryption algorithm ASCON can meet the performance and energy requirements implied by such workloads when implemented as a hardware accelerator.

1.1 Background

Vehicle-to-Everything (V2X) communication systems are important for modern intelligent transportation, enabling vehicles to exchange safety-critical information with infrastructure, vehicles, and network entities. Such communication supports applications including collision avoidance, cooperative awareness, and coordination. Since these applications are safety-critical, the V2X system must meet strict timing requirements. Standardization work on V2X services defines demanding latency requirements for message handling [1].

please add citations mentioned in the chat

In addition to low-latency requirements, V2X environments can generate a high volume of incoming messages, especially in dense traffic scenarios. This

places demands on system scalability and processing capacity. Previous studies show that cryptographic processing can become a major bottleneck in such environments, especially when a large number of messages must be authenticated or verified within a time frame [2], [3]. This means that the performance of a secure V2X system is influenced not only by the communication protocol itself, but also how efficiently the cryptographic operations can be executed under load.

Another important characteristic of V2X communication is that safety messages are small. Payload sizes range from approximately 50 to 300 bytes, depending on message content and communication context [4]. As a result, V2X systems are not constrained by raw data bandwidth alone. Instead, they must efficiently process a high number of short messages, making per-message overhead and latency especially important.

Hardware acceleration is therefore relevant in order to sustain high message rates without overloading general-purpose processors. Earlier work has shown that cryptographic performance depends not only on the selected algorithm, but also on the underlying implementation and hardware architecture [5]. In this context, field-programmable gate arrays (FPGAs) are ideal because they can provide both parallelism and energy efficiency. In particular, server-class FPGA platforms such as Xilinx Alveo U280 enable offloading computationally intensive cryptographic tasks in high-throughput environments.

1.2 Problem Statement

Existing research on V2X cryptographic performance primarily focuses on asymmetric cryptography, particularly elliptic-curve-based digital signatures for authentication. These studies show that cryptographic verification and related processing can limit scalability in dense traffic conditions [2], [3]. Much less attention has been given to symmetric authenticated encryption in the same settings.

ASCON, standardized through the NIST Lightweight Cryptography process, is a lightweight authenticated encryption algorithm intended to provide confidentiality and integrity at low implementation cost. While ASCON has been studied in hardware-oriented contexts, its suitability for V2X-related high message-rate workloads on a server-based FPGA platform remains unexplored. In particular, it is unclear whether an ASCON implementation can sustain V2X-relevant traffic conditions without introducing throughput bottlenecks, while also maintaining the strict low latency and good energy efficiency.

This is important since short-message workloads, which are typical in V2X communication [4], can shift the performance bottleneck from raw bandwidth to control overhead, cryptographic processing cost, and message latency. Therefore, a deployment-oriented evaluation is needed to determine whether ASCON is practically ready for implementation in a V2X system.

This thesis examines whether FPGA-based ASCON implementations can satisfy the strict throughput, latency, and energy-efficiency requirements of dense V2X communication environments.

1.3 Purpose and goals

The purpose of this thesis is to design and evaluate an ASCON-based hardware accelerator targeting deployment on the Xilinx Alveo U280 FPGA platform, with a focus on V2X-oriented workloads.

Rather than evaluating ASCON as a generic lightweight cryptographic primitive, this thesis investigates its behavior under conditions motivated by V2X communication, where a large number of short messages must be processed under strict timing constraints. The study focuses on throughput bottlenecks, message latency, and energy efficiency in order to assess practical deployment readiness.

The goals of the thesis are:

- to implement a functionally correct ASCON authenticated-encryption design on the Alveo U280 platform
- to verify correctness using standardized test vectors
- to define a V2X-relevant workload model based on message size and message rate
- to evaluate throughput and latency under short-message, high-traffic conditions
- to measure energy efficiency using metrics such as energy per message
- to assess whether the resulting implementation can be considered ready for deployment in a V2X-oriented system

To support evaluation, the goals above are formalized as a set of functional and non-functional requirements.

Functional requirements:

- The implementation must support ASCON-128 authenticated encryption and decryption in accordance with the NIST specification.
- The implementation must produce correct ciphertext and authentication tags, verified against official NIST Known Answers Test (KAT) vectors for ASCON-128.
- The implementation shall reject messages with corrupted authentication tags during decryption.
- The implementation must handle all messages from 0 to the configured maximum, including block-aligned lengths.

Non-functional requirements:

- The implementation must sustain a message rate of at least 4000 messages per second for message sizes of 50 to 300 bytes at 100 MHz.
- The end-to-end latency for a message shall not exceed 250 µs.
- Resource utilization shall be reported in terms of LUTs and flip-flops, allowing comparison with similar implementations.

1.4 Limitations

The scope of this thesis is limited as follows:

Evaluation is done entirely through simulation and post-implementation analysis in Vivado. No physical measurements are performed on deployed hardware. Energy values are estimates derived from switching activity rather than direct power measurements.

The implementation targets only a single-core design. Multi-core parallelism, which could increase message throughput proportionally, is discussed as future work but not implemented or presented in this thesis.

Side-channel and fault-injection countermeasures are not included. The thesis implements an unprotected baseline design.

Only the ASCON-128 variant is implemented and evaluated. Other ASCON variants, such as ASCON-128a or ASCON-Hash, are not considered.

1.5 Related Work

Previous research has investigated the performance of cryptographic mechanisms in V2X communication systems. Several studies show that cryptographic processing can be a limiting factor in busy traffic scenarios, where a large number of messages must be authenticated or verified within strict latency constraints. In particular, signature verification has been identified as a bottleneck affecting the system's scalability [2], [3].

In addition, studies on V2X communication highlight that message sizes are typically small, shifting performance considerations from raw bandwidth to per-message processing efficiency and latency [4]. These findings highlight the importance of evaluating cryptographic implementations under realistic workload conditions rather than relying solely on synthetic benchmarks.

Earlier scientific work has shown that ASCON lightweight authenticated encryption is well-suited for hardware implementation in resource-constrained environments, where performance and efficiency are essential objectives.

Studies have explored baseline hardware implementations of ASCON and evaluated performance characteristics. Kandi et al. present hardware implementations of ASCON and analyze different architectural design choices in terms of area usage, latency, and throughput. The results show that ASCON can be efficiently implemented in hardware and that implementation choices significantly impact performance and hardware complexity, making them relevant to FPGA-based designs. [6].

Another perspective is provided by Kaur et al., whose survey covers ASCON implementations across FPGA and ASIC platforms. The work summarizes results related to performance, hardware resource utilization, and power usage, and places ASCON within the broader context of lightweight cryptography. The survey highlights that design decisions strongly influence efficiency, energy characteristics,

and implementation overhead, motivating further focused evaluations of ASCON implementations on FPGA [7].

More application-oriented work has been demonstrated by Kurian and Chen, who implemented ASCON 128 on an FPGA for secure embedded and IoT systems. The implementation is evaluated for throughput, latency, and resource usage, demonstrating that ASCON can achieve high performance while maintaining a relatively low hardware footprint. This work serves as a direct reference for FPGA-based ASCON implementations and provides valuable comparison points for performance evaluation [8].

Kandi et al. investigate ASCON hardware designs that protect against side-channel and fault-injection attacks. The results show that these countermeasures significantly increase area usage and power consumption in both FPGA and ASIC implementations. Although these security-enhanced designs are outside the scope of the proposed project, the reported results provide insight into the trade-offs between security, performance, and energy consumption, and establish an essential baseline for unprotected implementations [9].

Together, these works form the scientific knowledge base for this project. They demonstrate that ASCON has been successfully implemented and evaluated in hardware, identify relevant performance and energy trade-offs, and motivate a focused FPGA-based study of baseline ASCON implementations targeting performance and energy evaluation.

Chapter 2

Technical Background

2.1 Lightweight Cryptography

Embedded and FPGA systems are commonly found in environments where computational power and energy usage are strictly constrained. In these systems, cryptographic operations need to meet real-time performance requirements while minimizing resource usage.

Traditional cryptographic algorithms are designed for general-purpose computing and may introduce unnecessary latency or hardware overhead when implemented in constrained environments. In response, Lightweight Cryptography (LWC) emerged as a solution that minimizes implementation complexity while preserving security guarantees.

In 2018, the National Institute of Standards and Technology (NIST) launched the Lightweight Cryptography competition to produce algorithms suitable for constrained devices. After an evaluation process, ASCON was selected in 2023 as the new NIST (LWC) standard [10].

2.2 Overview of ASCON

ASCON is a permutation-based cryptographic primitive that supports Authenticated Encryption with Associated Data (AEAD), hashing, and extendable-output functionality [8]. The design of ASCON aims to achieve a balance between security, speed, and hardware size, with a focus on minimizing implementation footprint.

ASCON is built around a single permutation operating on a fixed 320-bit internal state. This state is divided to five 64-bit words:

$$S = x_0 \parallel x_1 \parallel x_2 \parallel x_3 \parallel x_4$$

The state is divided into two parts:

- Rate (r): the outer portion where data is absorbed and extracted.
- Capacity (c): the inner portion that provides the security margin.

where c = 320-r

This structure allows ASCON to reuse the same permutation core for encryption, decryption, and hashing, eliminating the need for a separate key-scheduling algorithm. This design choice significantly reduces hardware complexity and memory requirements as discussed in the survey by Mirigladi et al. [11]

2.3 ASCON-128 AEAD Operation

The AEAD operation consists of four phases:

- Initialization
- Associated Data Processing
- Plaintext/Ciphertext Processing
- Finalization

For ASCON-128, the parameters are defined as:

- Internal state: 320 bits
- Key size: 128 bits
- Nonce size: 128 bits
- Rate **r** = 64 bits
- $p_a = 12$ permutation rounds (initialization and finalization)
- $p_b = 8$ permutation rounds (data processing)

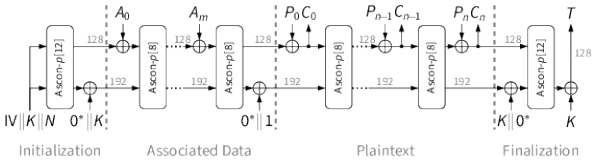


Figure 2.1: AEAD Structure Encrypt [12]

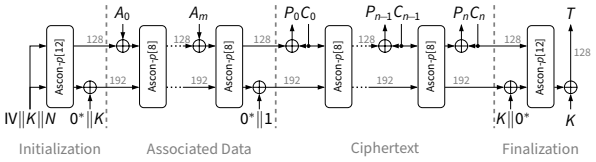


Figure 2.2: AEAD Structure Decrypt [12]

During initialization, the state is divided into initialization vector (IV), secret key, and nonce, followed by 12 permutation rounds to ensure full diffusion of secret key. Associated data and plaintext are absorbed block-wise into the rate portion of the internal state via XOR operations, with six permutation rounds applied between blocks. In the finalization phase, the key is re-inserted and another 12-round permutation is applied before extracting the authentication tag [13].

Because encryption and decryption both rely on the same permutation structure, an implementation can reuse a permutation module controlled by a finite state machine, simplifying datapath design and reducing area overhead.

2.4 Structure of the ASCON

The permutation function is the core of ASCON. Each round of the permutation consists of three layers.

2.4.1 Round Constant Addition

A constant unique to each round is XORed into register x_2. This step prevents structural symmetry across rounds and improves resistance against differential and linear cryptanalysis [11].

2.4.2 Substitution Layer

The substitution layer applies a 5-bit S-box bit-sliced across the five 64-bit state words. Each bit position across x_0 to x_4 forms a 5-bit vector processed by the S-box

The bit-sliced design has importance because:

- It avoids the use of memory-based lookup tables.
- it relies only on boolean operations.
- enables compact combinational logic implementations.

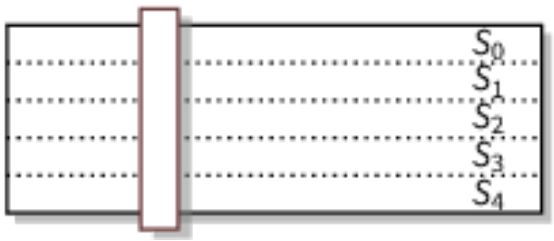


Figure 2.3: Illustration of vertical bit slicing in ASCON. Each highlighted column consists of one bit from each of the five 64-bit state words (S_0–S_4), forming a 5-bit input to the S-box [12]

2.4.3 Linear Diffusion Layer

The linear layer updates each 64-bit word using XOR and rotation operations for example:

$$S_0 := S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28)$$

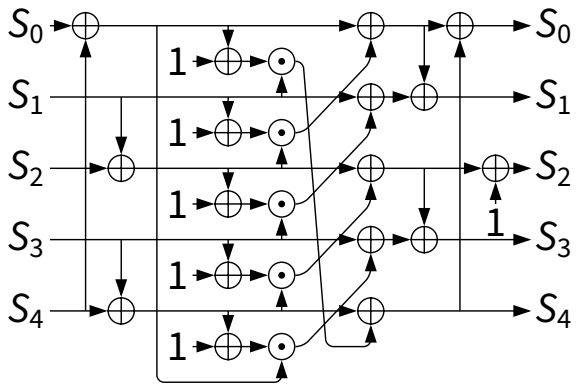


Figure 2.4: Bit-sliced 5-bit S-box applied to the columns of the five 64-bit state words [12]

This structure ensures diffusion of bit differences across each word while relying only on bitwise operations. Because rotations and xor are efficiently mapped to FPGA logic elements, this layer is hardware-friendly.

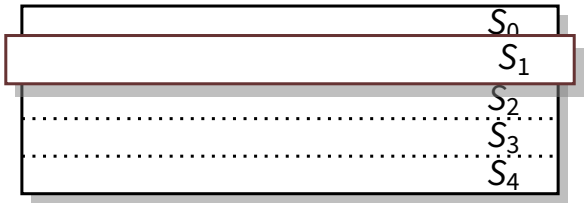


Figure 2.5: Linear diffusion layer: Each 64-bit word is XORed with two rotated versions of itself [12]

2.5 Hardware Evaluation Considerations

Hardware implementations of ASCON are usually evaluated using metrics such as resource utilization, operating frequency, throughput, and power consumption. Resource usage is measured in terms of lookup tables (LUTs) and flip-flops (FFs), which reflect the hardware cost of the implementation. Previous work has shown that architectural choices can significantly affect the stated metrics, leading to trade-offs between area, speed, and energy efficiency [11], [10].

Throughput is a key performance metric defined as the amount of data processed per second, expressed in Mbps or Gbps. In hardware implementations of ASCON, throughput depends on factors such as clock frequency and the number of cycles required to perform permutation rounds [14], [13]. Designs that reduce the number of cycles per operation or increase parallelism can achieve higher throughput with the trade-off of higher power consumption.

However, in application scenarios such as V2X communication systems, throughput alone is not sufficient to characterize performance. V2X systems are characterized by high message rates and small message sizes, where a large number of messages must be processed with strict timing constraints [2], [4]. In such cases, per-message latency becomes a critical measurement.

Latency is the time required to process a message, including the required permutation rounds and overhead. Even if a design achieves high throughput for large data streams, it may perform inefficiently when handling many short messages due to per-message overhead.

In addition to performance, energy efficiency is an important consideration in hardware implementation. Power consumption is measured directly, while energy efficiency is expressed as energy per bit. For short message workloads, energy per message provides a more relevant metric, as it reflects the cost of processing each individual message.

Overall, evaluating ASCON implementations requires the metrics throughput, latency, and energy efficiency. These metrics, when combined, provide a more complete understanding of system performance when considering deployment in high-volume message environments, such as V2X-oriented systems.

2.6 V2X Workload Characteristics

V2X communication systems are characterized by a high number of messages and relatively small message sizes. Safety messages range between 50 and 300 bytes and must be processed under strict latency constraints.

In dense traffic, a system may need to process thousands of messages per second. This creates a workload that tests the ability to efficiently process a large number of short messages.

This has important implications for hardware design. Architectures optimized for large data streams may perform inefficiently when processing small messages, due to control overhead and per-message latency.

Chapter 3

Method

3.1 Workload and Performance Targets

The evaluation in this work is based on a V2X-inspired workload model identified by its high rate of short messages. The system is made to process up to 4000 messages per second, corresponding to a maximum allowable processing time of 250 µs per message. The target of 4000 messages per second is based on requirements reported in previous V2X studies [3] and is therefore used as the workload model in this thesis.

Message sizes range from 50 to 300 bytes. Under these conditions, the data throughput remains low, and system performance is instead dominated by the efficiency of per-message processing.

Based on these characteristics, the evaluation focuses on latency per message, throughput, and energy efficiency. The objective is to determine whether the implementation can sustain the required message rate while maintaining acceptable latency.

3.2 Development Process

The implementation follows an iterative development process comprising three phases: baseline implementation, functional verification, and performance evaluation.

In the first phase, an encryption-only core variant was implemented based on the round-based ASCON-128 architecture described in Section 3.3. The goal was to have a functionally correct baseline before adding further complexity. The design was written in Verilog HDL and targeted synthesis using Vivado 2023.2.

In the second phase, the implementation was verified against NIST KAT vectors for ASCON-128, covering encryption and decryption across a range of messages. Verification was performed using a self-implemented testbench that compared outputs against the reference values and reported pass or fail for each vector. Failures in this phase guided design corrections. For example, KAT mismatches

for block-aligned message lengths required adding three new FSM states to correctly handle padding edge cases. Early integration of KAT verification ensured that extensions did not break existing functionality.

In the third phase, the verified design was used for performance evaluation. A separate testbench was designed to drive the core with repeated messages at the chosen sizes and to measure the cycle count for each operation. Post-implementation power analysis was later performed in Vivado using SAIF annotation files generated from this testbench, as described in Section 3.4.

This approach ensured correctness before performance ensured that all reported results were obtained from a verified and stable implementation. It reduced debugging complexity during the later SAIF-based power analysis flow.

3.3 Technical Design

The implementation architecture is structured around a hardware accelerator design for FPGA deployment. The system is designed around an ASCON processing core that executes permutation-based operations.

The design consists of three components: Input handling, the ASCON permutation core, and output handling. Input data is generated in the system and stored in internal registers before being handled by the permutation core. The core executes the required number of permutation rounds, after which the resulting data is sent to the output stage.

A round-based architecture is used, with a single permutation round implemented and reused across multiple clock cycles. The design choice reduces resource usage and enables predictable latency. Similar architectural trade-offs have been discussed in prior ASCON hardware implementations [14], [11], [6].

The control logic, implemented as a finite state machine, manages the algorithm's phases: initialization, processing, and finalization.

A high-level view of the architecture is illustrated in Figure 3.1, showing the interactions among the main components and the data flow through the system.

3.4 Evaluation Methodology

The performance of the implementation is evaluated using a set of predefined test cases designed to reflect the workload conditions.

Measurements are performed for multiple message sizes. Messages commonly range from 50 to 300 bytes. For each test case, the number of clock cycles required to process a single message is measured.

Latency is calculated by converting the cycle count to time using the operating frequency. Throughput is measured in bits per second.

Resource utilization is derived from synthesis reports and includes metrics such as LUTs, flip-flops, and the maximum operating frequency. Where possible,

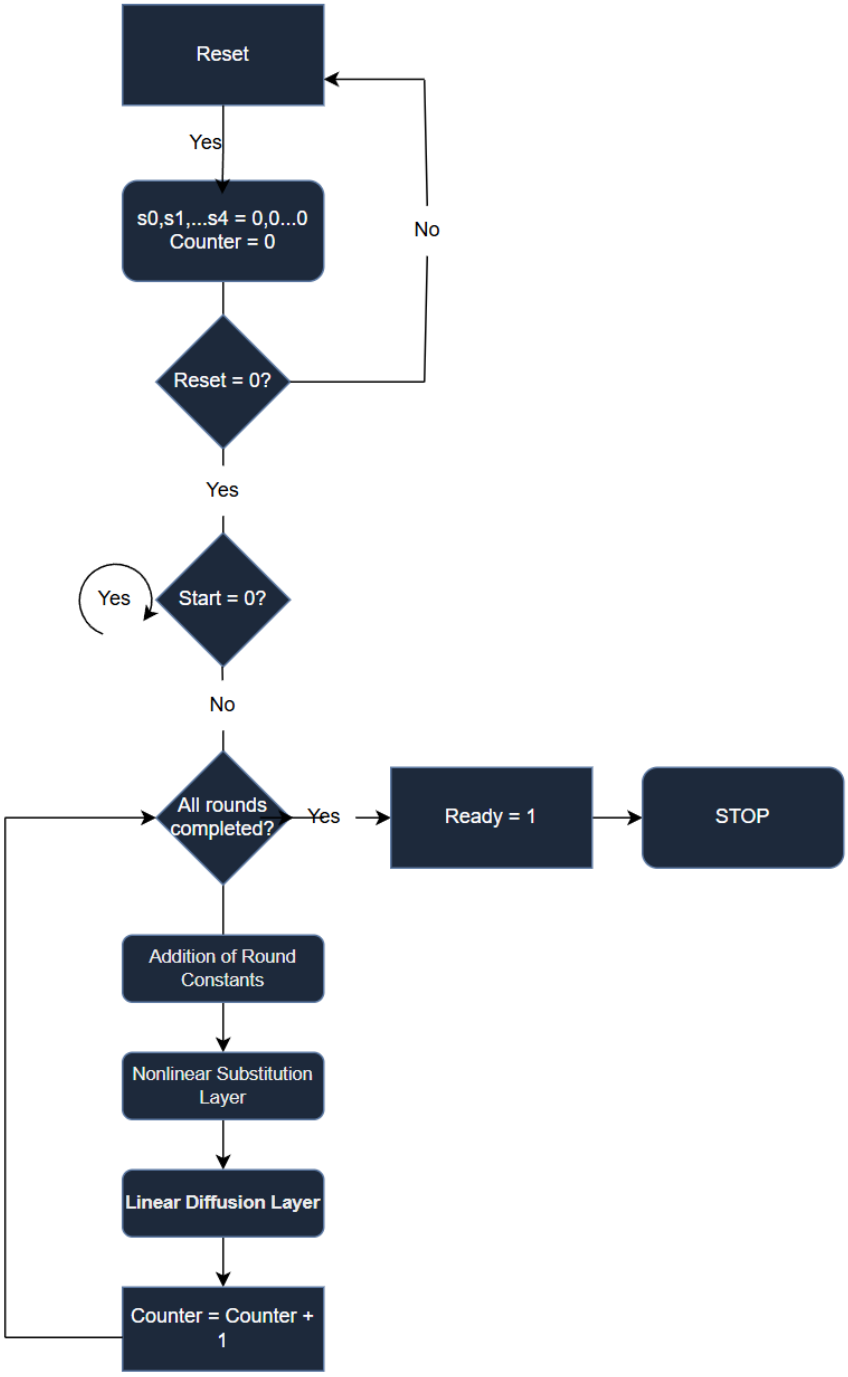


Figure 3.1: Simplified overview of the iterative ASCON round execution flow

power consumption is also measured or estimated, enabling evaluation of energy efficiency.

The evaluation approach follows commonly used metrics for hardware cryptographic implementations. The tests are made to determine whether the implementation can maintain the target message rate of 4000 messages per second while keeping latency within acceptable bounds.

3.4.1 Performance metrics

The implementation was evaluated using latency, throughput, message rate, and energy efficiency metrics.

Latency was calculated from the measured cycle count and operating frequency as:

$$t_{latency} = \frac{N_{cycles}}{f} \tag{3.1}$$

Where N_{cycles} is the number of clock cycles required to process a message and f is the clock frequency.

Throughput was calculated accordingly:

$$Throughput = \frac{Message\ Size\ (bits) \cdot f}{N_{cycles}} \tag{3.2}$$

Message rate was calculated as the inverse of latency:

$$Message\ Rate = \frac{1}{t_{latency}} \tag{3.3}$$

Energy per message was calculated using the estimated total on-chip power and measured latency:

$$E_{message} = P_{total} \cdot t_{latency} \tag{3.4}$$

where P_{total} is the estimated total on-chip power (W). $t_{latency}$ is the message latency (S) and $E_{message}$ is the energy per message (J).

Lastly, energy per bit was calculated as:

$$E_{bit} = \frac{E_{message}}{Message\ Size\ (bits)} \tag{3.5}$$

3.4.2 Power Estimation

Power consumption is estimated using Vivado's post-implementation power analysis tool. To obtain accurate switching activity data, a Switching Activity Interchange Format (SAIF) file is generated from a post-synthesis functional simulation of the design under the same workload used for performance evaluation. The

SAIF file keeps track of the toggle rate of each signal in the design and is used to annotate the power analysis.

The SAIF annotation is created at the post-implementation netlist level using the `report_power` command in Vivado, with the strip path set to match the simulation hierarchy.

Total on-chip power is the sum of static and dynamic power. Logic power, reflecting the ASCON core's switching activity, is extracted separately to enable platform-independent comparison of core energy consumption. Energy per message is later calculated as:

$$E_{message} = P_{total} \cdot t_{latency} \tag{3.6}$$

P_{total} is the total on-chip power and $t_{latency}$ is derived from equation 3.1 being the message latency.

To validate the results, post-synthesis and post-implementation results are compared. Similar values produced by both tests, providing confidence in the reported results. Results are reported for both the Artix-7 and the Alveo U280 to showcase the difference between a resource-constrained FPGA and a server-class FPGA deployment.

3.5 Reproducibility

The methodology is intended to enable a reader with a background in hardware design to reproduce the results. The workload assumptions, architectural choices, and evaluation procedure are explicitly defined and can be followed.

Implementation details, such as design structure, configuration parameters, and toolchain usage, are further described in the implementation chapter.

Chapter 4

Implementation

This chapter showcases the hardware implementation of the ASCON-128 authenticated encryption algorithm in Verilog HDL, targeting the Xilinx Alveo U280 FPGA platform. The implementation is a round-based permutation architecture controlled by a finite-state machine (FSM) that manages the full AEAD encryption pipeline, including initialization, associated data processing, plaintext encryption, and finalization.

4.1 Module Structure

The implementation consists of two Verilog modules: the top-level `ascon_core` module and a submodule `ascon_round`. The top-level module is parametrized by `MAX_PT_BYTES` and `MAX_AD_BYTES`, which define the maximum supported plaintext and associated data sizes, respectively. This parametrization allows the design to be flexible for different V2X message sizes without altering the core logic.

The `ascon_core` module takes as inputs the 128-bit key, the 128-bit nonce, the plaintext data, and the associated data, along with their respective byte lengths. It produces the encrypted ciphertext, a 128-bit authentication tag, and a done signal as outputs. All input data is registered internally at the start of each encryption operation, ensuring stable operation across clock cycles.

4.2 `Ascon_round` Submodule

The `ascon_round` module implements a single round of the ASCON permutation as a combinational circuit. It takes the five 64-bit state words x0 to x4 and a round constant rc as inputs, and produces updated state words as outputs in a clock cycle. This combinational design is structured so that there are no registers inside the round module itself; state is held in the top module `ascon_core` registers and updated on each rising clock edge.

Each round consists of the three standard ASCON layers:

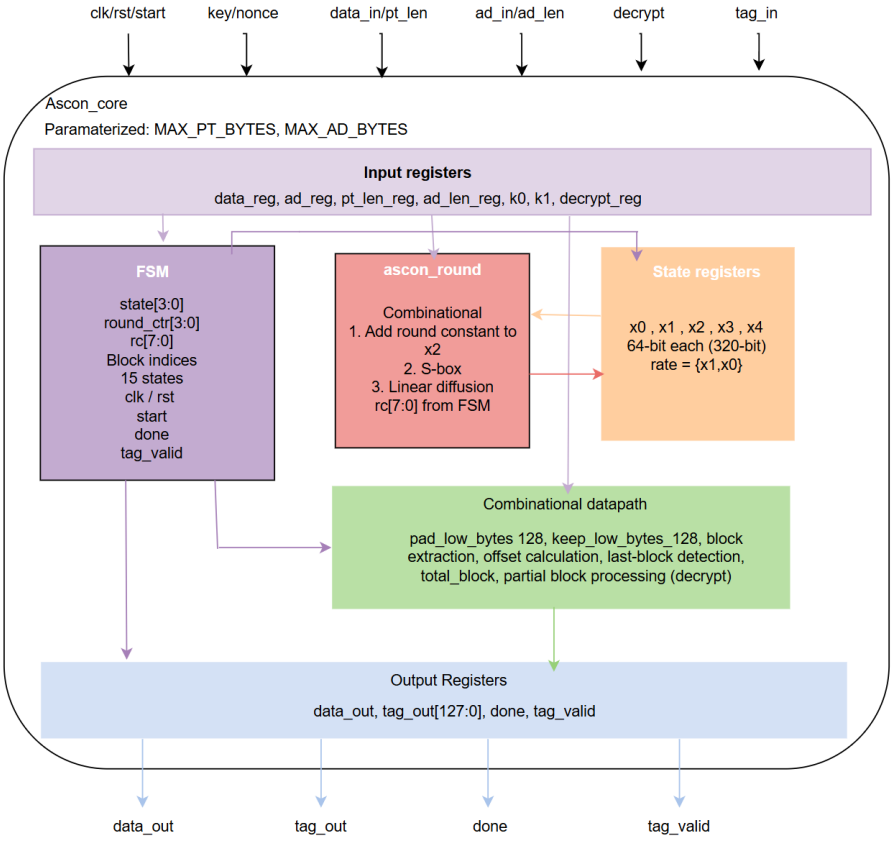


Figure 4.1: Architecture overview of the ASCON core, showing the FSM, state registers, permutation round, and datapath components.

- **Round constant addition** - round constant rc is XORed into state word x2, introducing asymmetry between rounds.
- **Substitution layer** - 5-bit S-box is applied in a bit-sliced manner across all five 64-bit state words. The implementation uses only Boolean, AND, OR, XOR, and NOT operations, avoiding any lookup tables and enabling fast combinational logic synthesis.
- **Linear diffusion layer** - state word is XORed with two rotated versions of itself using the rotr64 function. For example: $x_0 \oplus (x_0 \gg 19) \oplus (x_0 \gg 28)$. The rotation amounts are different per word, as specified by the ASCON standard.

By dividing the round function into its own module, the design achieves a clean separation between control logic and data path logic. The FSM in `ascon_core` initiates the same `ascon_round` on every clock cycle, iterating through the required number of rounds.

4.3 Helper module (`ascon_helpers.vh`)

Shared functions used across the design are collected in a separate Verilog include file, `ascon_helpers.vh`. Separating these functions from the top module keeps the core logic readable and makes the functions reusable without duplication across files. The file defines five functions:

- **`get_p12_rc`** - return the 8-bit constant for a given round index in the 12-round permutation p12. Constants are defined as a combinational case statement and range from 0xf0 at round 0 to 0x4b at round 11.
- **`get_p8_rc`** - returns the round constant for the 8-round permutation p8, used during associated data and plaintext processing. The values are a subset of p12, starting at 0xb4.
- **bswap64** - reverses byte order of a 64-bit word. Applied to the key and nonce on input, and to the tag words on output, to align with the big-endian byte ordering required by the ASCON specification.
- **`keep_low_bytes_128`** - masks 128-bit value to retain only the lowest nbytes bytes, zeroing the remains. Used to strip undefined bytes from the ciphertext output of a partial plaintext block.
- **`pad_low_bytes_128`** - performs the same byte masking as `keep_low_bytes_128` but on top of that inserts a 0x01 byte after the last valid byte. This implements the ASCON padding rule applied to partial associated data and plaintext blocks.

4.4 Finite State Machine

The control flow is implemented as a 15-state finite state machine (FSM), including states to handle padding and edge cases. Figure 4.2 shows the complete state

transition diagram.

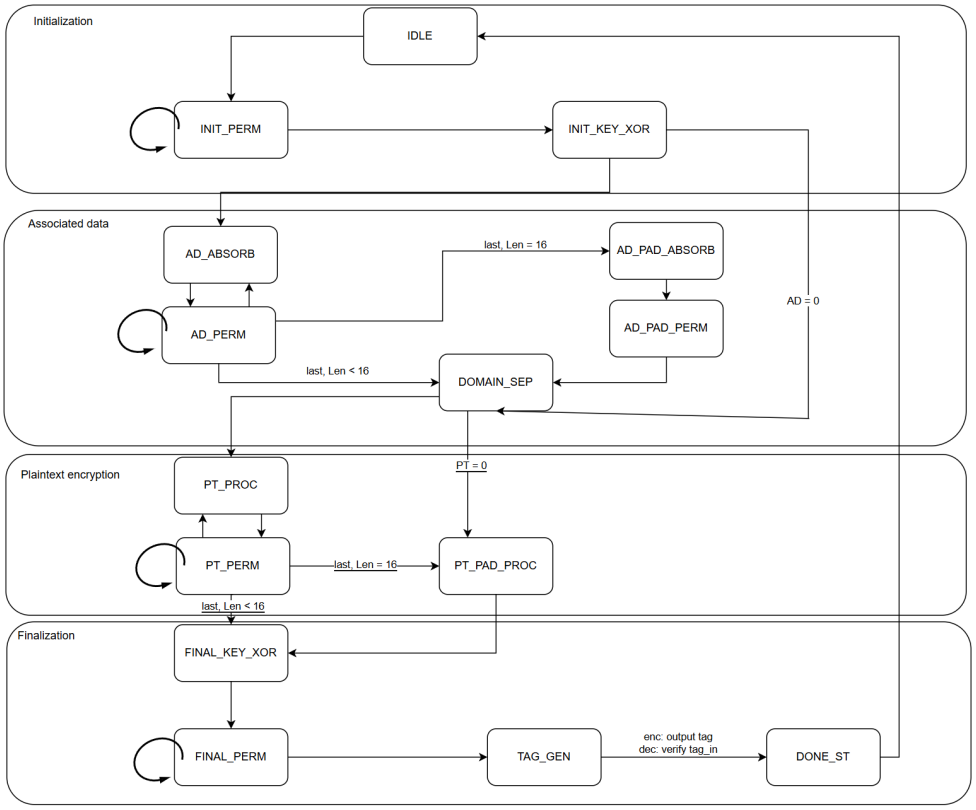


Figure 4.2: Control flow of the ASCON encryption process, including initialization, associated data processing, plaintext encryption, and finalization.

The FSM states and their tasks are as follows:

- **IDLE** - Waiting for start signal. On assertion, all input data is stored into internal registers, and the ASCON state is initialized with the IV, KEY, and nonce. The key is byte-swapped using **bswap64** to match the ASCON specification's endianness.
- **`INIT_PERM`** - Executes the p12 permutation (12 rounds) on the initial state by running `ascon_round` twelve times, applying round constants from `get_p12_rc`.
- **`INIT_KEY_XOR`** - XORs the key split into two words k0 and k1 into the lower state words x3 and x4, completing the initialization phase as specified by ASCON-128.
- **`AD_ABSORB`** - Absorbs 128-bit block of associated data into the rate portion via XOR, with ASCON padding applied to the final block using `pad_low_bytes_128`.
- **`AD_PERM`** - Runs p8 (8 rounds) after each associated data block using round constants from `get_p8_rc`.
- **`DOMAIN_SEP`** - Applies the separation bit by XORing a constant into state x4, distinguishing the associated data phase from the plaintext phase.

- **PT_PROC** - Depending on the mode data is processed accordingly. For encryption (decrypt = 0), the output is computed as rate XOR plaintext and the state is updated with the resulting ciphertext. For decryption (decrypt = 1), the output is computed as rate XOR ciphertext, producing plaintext. For full blocks, the state is updated directly with the ciphertext. For partially filled blocks, only the valid bytes are kept with the function `keep_low_bytes_128`, while the state update uses `pad_low_bytes_128` to maintain correct sponge behavior.
- **PT_PERM** - Runs p8 between plaintext blocks, identical to `AD_PERM`.
- **AD_PAD_ABSORB** - Handles cases where associated data length is precisely aligned to a full 16-byte block. An additional padding-only block is absorbed into the state.
- **AD_PAD_PERM** - Executes the p8 permutation after the additional associated-data padding from `AD_PAD_ABSORB`.
- **PT_PAD_PROC** - Handles the case when the plaintext length is zero or exactly aligned to a full 16-byte block. A padding block is processed before finalization.
- **FINAL_KEY_XOR** - XORs the key in the finalization phase before the encryption round is done, as required by the ASCON-128 specification.
- **FINAL_PERM** - Executes the last permutation to produce the finalized state.
- **TAG_GEN** - Computes the 128-bit authentication tag by XORing the key into states x3 and x4 and byte-swapping the results. In encryption mode, the tag is output directly. In decryption mode, the computed tag is compared with the input tag `tag_in`, and the result is reported through the `tag_valid` signal.
- **DONE_ST** - Outputting a high signal signifying that the encryption is done, waiting for the next input, transitions back to IDLE.

4.5 Round Constant Generation

Round constants are managed by two Verilog functions, `get_p12_rc` and `get_p8_rc`, implemented as combinational case statements in the helper file `ascon_helpers.vh`. As mentioned in the function names, the p12 constant spans rounds 0 to 11, starting at 0xf0 and decrementing by 0x0f per round. The p8 constants are a subset, starting at the same position as round 4 of p12 (0xb4). This matches the ASCON specification and ensures correct cryptographic behavior.

4.6 Datapath and Block Processing

The datapath computes all intermediate signals as wire assignments, evaluated together with the FSM. The current associated data and plaintext blocks are extracted from the input registers using variable-offset bit slicing, for example `data_reg[(pt_block_idx * 128) +: 128]`. Block indices are tracked by `ad_block_idx` and `pt_block_idx`,

p^{12}	p^8	Constant c_r	p^{12}	p^8	Constant c_r
0		00000000000000f0	6	2	0000000000000096
1		00000000000000e1	7	3	0000000000000087
2		00000000000000d2	8	4	0000000000000078
3		00000000000000c3	9	5	0000000000000069
4	0	00000000000000b4	10	6	000000000000005a
5	1	00000000000000a5	11	7	000000000000004b

Table 4.1: ASCON round constants for p^{12} and p^8 permutations.

which are incremented as each block is processed.

The number of 16-byte blocks for both associated data and plaintext is computed using ceiling division: (len + 15) » 4. Remaining byte counts and last-block flags are derived from these values, allowing correct handling of partial final blocks without requiring separate control logic.

Two helper functions defined in `ascon_helpers.vh` are used to support partial-block processing. The function `keep_low_bytes_128` zeroes all bytes beyond a stated byte count, used to strip undefined bytes from the ciphertext output of a partial block. The function `pad_low_bytes_128` performs the same masking but additionally inserts a 0x01 byte after the last valid byte, implementing the ASCON padding rule for partial blocks.

4.7 Endianness and Key Handling

The ASCON specification defines its state words in big-endian byte order. Verilog naturally operates in a bit-indexed manner with no inherent byte ordering. The bswap64 function is applied to the key and nonce words at initialization time and to the tag words at output time. This reversal ensures that the internal state representation matches the reference implementation, which is critical for correctness verification against the standard test vectors.

4.8 Design Rationale

The design choice of a round-based iterative architecture, rather than a fully unrolled or pipelined design, reflects the deliberate trade-off between resource usage and throughput. A fully unrolled design would consist of twelve or eight separate round modules in sequence, increasing LUT and flip-flop usage. The iterative approach reuses a single `ascon_round` instance across all rounds, minimizing hardware footprint.

This architecture provides predictable, deterministic latency per message, which is straightforward to calculate from the number of rounds per phase, the number of data blocks, and the operating clock frequency. For the V2X workload model

fixed in Section 3.1, where messages in the range from 50 to 300 bytes, the per-message cycle count remains low enough that the 250 µs latency bottleneck is comfortably met even at moderate clock frequencies, as reported in Chapter 5.

Separating the permutation logic into the `ascon_round` submodule also simplifies verification, as the round function can be tested independently before integration into the full pipeline. This supports the iterative development approach mentioned in Section 3.2.

Chapter 5

Results

This chapter presents the results obtained from the ASCON-128 hardware implementation. Results are divided into five sections: Functional correctness verification, resource usage, latency and throughput performance, power estimation, and energy efficiency. All performance measurements were performed at an operating frequency of 100 MHz using simulations. Power estimation was performed using Vivado's post-implementation power analysis flow with Switching Activity Interchange Format (SAIF) annotations on two target platforms: the Artix-7 FPGA and the Xilinx Alveo U280.

5.1 Functional Correctness Verification

Prior to evaluation, the implementation was tested for functional correctness using the official ASCON-128 known test answers (KAT) published alongside the NIST standardization submission. The test consisted of providing inputs such as the reference key, nonce, associated data, and plaintext to the `ascon_core` module in simulation, then comparing the produced output of ciphertext and authentication tag against the expected reference outputs.

Verification was executed in Vivado using a behavioral simulation testbench. For each test, three checks were performed: encryption correctness (ciphertext and tag matching the NIST reference), decryption correctness (recovering the plaintext from ciphertext and the valid tag), and rejection of a corrupted tag (`tag_valid = 0` when the tag is modified). The implementation produced outputs identical to the NIST reference vectors for both encryption and decryption, and correctly rejected all corrupted tags.

5.2 Resource Utilization

Resource utilization was obtained from the post-implementation reports generated by Vivado for both target platforms. Table 5.1 presents the Look Up tables (LUT) and flip-flop counts for Artix-7 and Alveo U280 implementations.

Table 5.1: Post-implementation resource utilization on Artix-7 and Alveo U280.

Metric	Artix-7	Alveo U280
LUTs	988	954
Flip-Flops (FFs)	618	617

Resource usage is almost identical for both platforms, with a difference of 34 LUTs and one flip-flop. The total LUT count is below 1000, and the flip-flop count is below 620 for both platforms.

5.3 Latency and Throughput

Performance metrics were measured by simulating the `ascon_core` module at 100 MHz across six message sizes representative of the different V2X workloads. For each message, the number of clock cycles from the start signal to the done signal was recorded. Given identical RTL designs on both platforms and the same clock frequency, the cycle counts, latency, and throughput values are platform-independent. Table 5.2 summarizes the measured performance metrics.

Table 5.2: Measured performance at 100 MHz. Cycle counts are platform-independent.

Msg Size (bytes)	Cycles	Latency (µs)	Throughput (Mbps)	Msg Rate (msg/s)
16	59	0.59	216.95	1,694,915
20	60	0.60	266.67	1,666,667
64	87	0.87	588.51	1,149,425
128	123	1.23	832.52	813,008
320	231	2.31	1,108.23	432,900
400	276	2.76	1,159.42	362,319

Latency ranges between 0.59 µs and 2.76 µs. Throughput ranges between 216.95 Mbps and 1159.42 Mbps. Message rate ranges between approximately 1,694,915 and 362,319 messages per second.

5.4 Power Estimation

Power usage was estimated using Vivado post-implementation power analysis flow on both the Artix-7 and Alveo U280 platforms. Switching activity was tracked using SAIF files generated from post-synthesis functional simulation. The implemented testbench executes a loop of encryption followed by decryption, producing switching activity similar to a V2X system. Annotation coverage reached 100 percent of design nets on both platforms, giving high confidence in both estimates. Table 5.3 presents the power breakdown

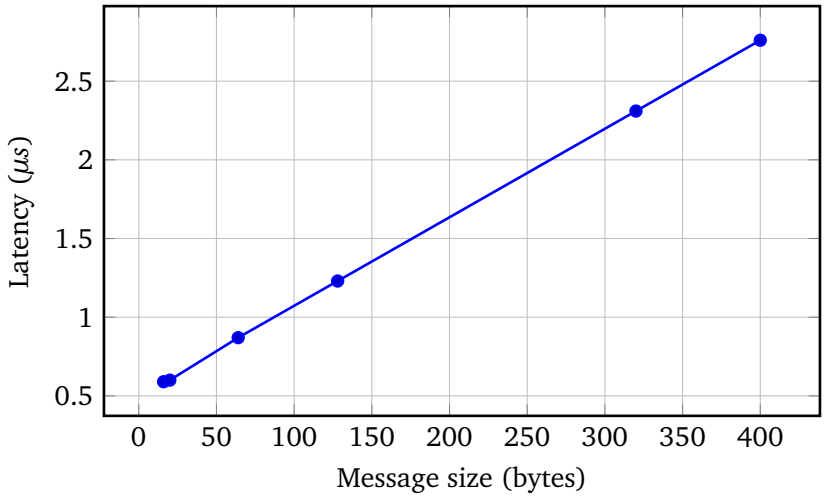


Figure 5.1: Latency as a function of message size at 100 MHz.

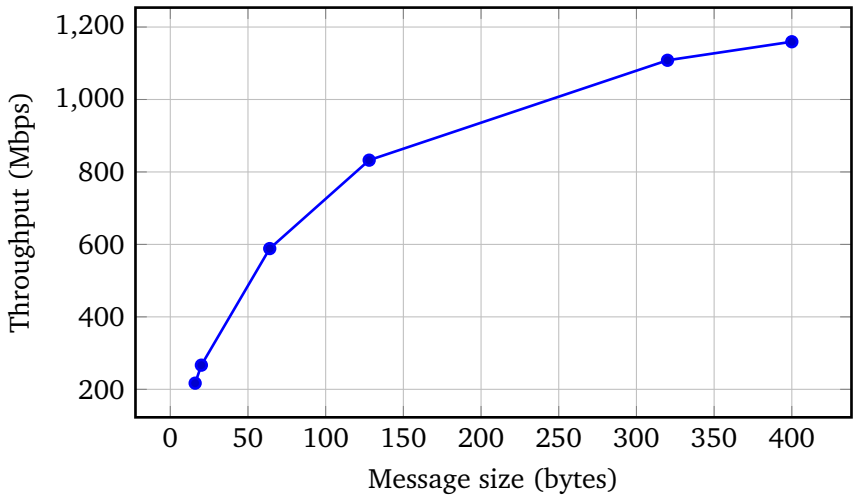


Figure 5.2: Throughput as a function of message size at 100 MHz.

Table 5.3: Post-implementation power estimation on Artix-7 and Alveo U280.

Power Component	Artix-7 (W)	Alveo U280 (W)	Note
Total On-Chip	0.173	3.226	
Device Static	0.131	3.130	Platform overhead
Dynamic (total)	0.042	0.096	
Logic power	≈ 0.007	≈ 0.007	Core, identical on both

5.5 Energy per Message

The energy per message formula is total on-chip power and measured latency $E = P_{\text{total}} \cdot t_{\text{latency}}$. Table 5.4 presents energy per message for the two platforms across all message sizes.

Table 5.4: Energy per message for Artix-7 and Alveo U280, calculated as total on-chip power × latency.

Msg Size (bytes)	Energy Artix-7 (μJ)	Energy U280 (μJ)	Ratio (U280/Artix-7)
16	0.102	1.906	$\approx 18.7\times$
20	0.104	1.938	$\approx 18.6\times$
64	0.151	2.810	$\approx 18.6\times$
128	0.213	3.973	$\approx 18.7\times$
320	0.400	7.461	$\approx 18.7\times$
400	0.477	8.914	$\approx 18.7\times$

Energy per message on the Artix-7 ranges from 0.102 µJ to 0.477 µJ. The corresponding values on the U280 range from 1.906 µJ to 8.914 µJ.

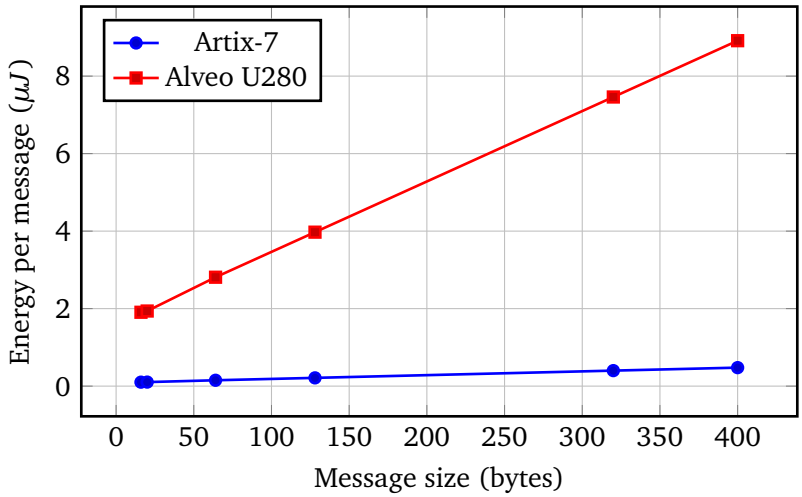


Figure 5.3: Energy per message for Artix-7 and Alveo U280 across evaluated message sizes.

Table 5.5: Requirement Verification Summary

Requirement	**Verification Method**	**Result**	**Status**
ASCON-128 encryption and decryption support	Verification using official NIST KAT vectors	Correct ciphertext, plaintext, and authentication tags produced	Passed
Corrupted authentication tags rejected	Decryption test with modified authentication tags	Invalid tags correctly rejected with `tag_valid = 0`	Passed
Support block-aligned and variable message lengths	KAT verification and edge-case simulation	Padding edge cases handled correctly after FSM extension	Passed
Sustain at least 4000 messages/s for V2X workloads	Cycle-count measurements at 100 MHz	Minimum measured throughput was 362,319 msg/s (400 bytes)	Passed
Keep latency within V2X timing constraints	Latency derived from measured cycle counts	Maximum measured latency was 2.76 μs	Passed
Report FPGA resource utilization	Vivado post-implementation utilization reports	Artix-7: 988 LUTs / 618 FFs	Passed
Estimate power and energy consumption	Post-implementation SAIF-based power analysis	Logic power estimated at approximately 7 mW	Passed

Chapter 6

Discussion

This chapter discusses the results presented in Chapter 5 in relation to the research questions stated in Chapter 1 and the V2X workload model from Chapter 3. The discussion covers performance, throughput behavior, resource utilization, power and energy efficiency, the validity of the methodology, the development process, and societal implications.

6.1 Performance Analysis

The core research question of this thesis is whether an ASCON-128 hardware implementation can meet the performance requirements of a V2X-oriented system processing up to 4000 messages per second. The results show that the implementation exceeds the set target across all evaluated message sizes, with margins ranging from 90 times at 400 bytes to 423 times at 16 bytes. The V2X workload model, motivated by ETSI service requirements [1], defines messages of 50 to 300 bytes at up to 4000 msg/s. Earlier work has identified cryptographic processing as a potential bottleneck in dense traffic scenarios [2], [3]. The results of this work show that this bottleneck does not arise for ASCON-128 at 100 MHz, and that the cryptographic processing latency of 0.59 to 2.76 µs (Figure 5.1) increases linearly with message size. This behavior is expected for iterative round-based architecture.

These findings are consistent with prior hardware implementation of ASCON. Kurian and Chen [8] implemented ASCON-128 on an FPGA and demonstrated comparable throughput, concluding that ASCON achieves high performance with a low hardware footprint. Kandi et al. [6] also show that architectural choices affect performance, as observed in the throughput scaling with message size in this work. The survey by Kaur et al. [7] confirms that throughput depends on clock frequency and cycles per permutation round, consistent with the results of this thesis.

To position the proposed design with respect to existing implementations, Table 6.1 presents a direct comparison with selected state-of-the-art ASCON-128 designs on Artix-7 FPGA platforms. The reported throughput corresponds to the

400-byte case, representing steady-state performance in which the initialization overhead is at its lowest.

Table 6.1: Comparison of ASCON-128 implementations on Artix-7 FPGA

Design	LUTs	FFs	Freq (MHz)	Throughput (Mbps)	Power (mW)	Mbps/LUT
This work	988	618	100	1159	173	1.17
Alharbi et al. [10]	1756	912	317	376	222	0.21
Raj et al. [15]	1330	870	107	457	31	0.34
Ahmet [16]	2957	595	55.25	3535.91	N/A	1.19

As shown in Table 6.1, the proposed implementation attains higher throughput than Alharbi et al. and Raj et al., while using fewer LUT resources. Compared to Ahmet, the throughput is lower, but at a lower area cost.

When normalized by area, the efficiency (Mbps/LUT) is comparable to Ahmet's and higher than the other designs.

6.2 Throughput Behavior and Architecture Trade-offs

The throughput scaling, low for small messages and converging towards 1.16 Gbps for larger inputs, is a fundamental property of sponge-based AEAD constructions. As shown in Figure 5.2, throughput increases with size and converges toward 1.16 Gbps. This behavior occurs because the fixed initialization and finalization overhead becomes less dominant as the amount of processed plaintext increases. Mirigaldi et al. [11] discuss this topic, noting that the fixed overhead of initialization and finalization dominates for short messages. This is also evident in this work's results: at 16 bytes, the 12-round initialization and finalization phases account for the majority of the total cycles, whereas at 400 bytes their contribution is much smaller. For V2X messages of 50 to 300 bytes [4], the design achieves throughput of 500 Mbps to 1.1 Gbps, exceeding all V2X data rate requirements.

The iterative architecture trades maximum throughput for reduced resource utilization. Kandi et al. [6] discuss this as a trade-off in ASCON hardware design, saying that iterative design achieves lower area than fully unrolled or pipelined alternatives. In this case, the iterative design already exceeds the V2X target by two orders of magnitude, the iterative architecture is the logical choice for this deployment context.

6.3 Resource Utilization

The design consumes 988 LUTs and 618 flip-flops on Artix-7, and 954 LUTs and 617 flip-flops on the U280, a difference of 34 LUTs between platforms, showing that the same RTL was synthesized on both devices. The figures are consistent with iterative ASCON implementations reported in the literature [7]. More complex

designs, such as side-channel-protected implementations [9], show higher area overhead, providing a reference for the cost of additional security features.

6.4 Power and Energy Efficiency

The total on-chip power is 0.173W on Artix-7 and 3.226W on the U280, with the difference largely being static power (0.131W versus 3.130W). The U280's large infrastructure introduces a significant static power baseline that is independent of the implemented design. The logic power associated with the ASCON core switching activity is 7mW on both platforms, confirming that the cryptographic computation itself consumes similar energy.

The energy per bit on Artix-7 is 149 pJ/bit at 400 bytes, which falls within the expected range for lightweight cryptographic implementation. The U280 energy per message is higher at 8.914 µJ for 400 bytes, driven entirely by total device power rather than differences in core logic. This demonstrates the importance of platform selection when targeting energy-efficient deployments as displayed in Figure 5.3, energy per message increases with message size on both platforms because larger messages require more processing cycles and therefore longer execution time.

6.5 Validity of Results

The performance results obtained in Chapter 5 are based on simulation rather than hardware measurement. Cycle counting in simulation is a deterministic, exact method for this type of RTL design: the number of cycles per operation is fully predictable from the FSM logic and input parameters, and the simulation results are close to identical to what would be observed in hardware at the same clock frequency. The throughput and latency values are, for this reason, reliable for the evaluated operating conditions.

The power estimation approach used in this work is based on post-implementation analysis with SAIF annotations from post-synthesis functional simulation is a standard approach when direct hardware power measurements are unavailable. In this work, 100% annotation coverage was achieved across all design nets on both platforms, and Vivado reported high confidence for both estimates. To further validate the results, post-synthesis and post-implementation results were compared: the Artix-7 post-synthesis estimate was 0.175W, and the post-implementation estimate was 0.173W, a difference of 1.2%. This close agreement between two independent estimations supports the reliability of the reported power values.

An important limitation was that power measurements on the Alveo U280 are dominated by static power due to its large FPGA architecture. This makes it difficult to isolate the ASCON core's energy consumption.

To address this, results from a smaller board Artix-7 are also included. On this platform, the contribution of static power is noticeably lower, providing a clearer

view of the core's actual power usage and energy efficiency.

The use of two platforms therefore serves two purposes: the U280 reflects deployment on a high-performance system, while the Artix-7 provides a representative characterization of the core in a lightweight embedded context. In addition, the Artix-7 platform enables a fair comparison with state-of-the-art ASCON hardware implementations, which are usually evaluated on resource-constrained FPGA devices.

Another limitation is that the testbench repeatedly encrypts and decrypts a fixed message pattern. Real-world V2X traffic would yield varied message content, leading to slightly different switching activity patterns. However, for the purpose of estimating steady-state power consumption, a repeating workload is used to approximate it.

The functional correctness of the implementation was verified against official KAT vectors for ASCON-128, covering both encryption and decryption with correct and corrupted tags. Matches were obtained for all tested vectors. This provides strong grounds that the cryptographic behavior of the implementation is correct in line with the reference specification.

6.6 Development Process

The development process had an iterative approach, as mentioned in Section 3.2, beginning with a functionally correct encryption-only baseline before adding support for decryption and additional edge cases. This ordering showed positive results: verifying correctness against NIST test vectors early on in the process allowed later extensions to go more smoothly. Including decryption, tag verification, and padding edge-case handling to ensure the core cryptographic behavior remained intact.

The most significant challenge encountered during the implementation was correctly handling the padding edge cases for block-aligned AD and PT lengths. The ASCON specification requires an extra padding block when the associated data or plaintext length is an exact multiple of the 16-byte block size. This issue was addressed by adding three new FSM states: `AD_PAD_ABSORB, AD_PAD_PERM, AD_PAD_PROC`, which are not present in the initial design. These states were added after KAT verification revealed tag mismatches for block-aligned test vectors. This experience highlights the importance of testing against edge-case vectors early in the verification process.

A second challenge was encountered during the power estimation flow. Achieving 100% SAIF annotation coverage required iterations of the testbench and wrapper design. An issue early on was a naming collision between two instances both named **dut** in the module hierarchy, which caused `Vivado's strip_path` annotation to match signals at the wrong level, resulting in poor annotation coverage. Resolving this required restructuring the wrapper hierarchy and using distinct instance names at each level. In hindsight, establishing a clear module naming convention from the beginning would have avoided this issue entirely.

For alternative approaches, a pipelined architecture would achieve higher throughput and lower per-message latency by allowing new messages to enter the permutation pipeline before the previous is completed. However, given that the iterative design already exceeds the V2X target by a factor of 90-423, pipelining would provide no benefit for this application. A more relevant discussion would be a multi-core design in which several `ascon_core` instances operate in parallel, increasing the aggregate message rate proportionally. Completing implementation on the actual Alveo U280 with a full host interface, including the host communication interface (PCIe) and data transfer mechanisms (DMA) transfers, would provide a more complete end-to-end system evaluation and allow direct measurement of system-level latency.

6.7 Societal Aspects

The outcomes of this work are relevant to the deployment of secure V2X systems in real-world transportation. V2X communication systems are core facilitators of intelligent transport infrastructure, supporting innovations such as collision avoidance, cooperative driving, platooning, and emergency vehicle coordination. The mentioned applications are safety-critical: incorrect processing of a cryptographic message could have consequences for pedestrians and drivers. Ensuring that the cryptographic processing layer in such a system is fast and energy-efficient is therefore directly linked to transportation safety.

The use of ASCON as the cryptographic algorithm is endorsed by its standardization through the NIST lightweight cryptography process, completed in 2023. This implies that the algorithm has undergone public inspection and formal security analysis by the cryptographic research communities. Using a standardized, publicly reviewed algorithm is preferable to closed-source solutions, as it enables independent verification of security and reduces the risk of vulnerabilities in closed or undocumented designs. This work therefore contributes to the case for adopting standardized lightweight cryptography in V2X systems.

From an environmental perspective, the energy efficiency demonstrated by the ASCON implementation is applicable to the sustainability of V2X infrastructure. Roadside units and vehicle systems that continuously manage large volumes of cryptographic messages consume energy. A low-power cryptographic core reduces the contribution, supporting broader goals of reducing energy use in the transportation sector. The measured logic power of approximately 7 mW for the ASCON core represents a small energy cost per message, which scales well even at high message rates.

No negative societal or ethical consequences have been identified for this work. The implementation does not introduce new attack surfaces or impair existing security properties. The design supports authentication and confidentiality for V2X messages, thereby strengthening rather than compromising the security of the communication layer. The use of open, standardized algorithms and public documentation supports transparency and reproducibility, which are important

principles in engineering work with safety implications.

Chapter 7

Conclusion

This thesis's main objective was to determine whether the ASCON-128 lightweight authenticated encryption algorithm can meet the performance requirement of a V2X-oriented system, with a specific target of processing up to 4000 messages per second across message sizes of 50 to 300 bytes. The results show that this requirement is satisfied with a large margin.

A functionally correct ASCON-128 AEAD core was implemented in Verilog HDL, supporting encryption and decryption. The design follows a round-based, iterative finite-state machine with 15 states, including three additional states introduced to correctly handle block-aligned padding edge cases in accordance with the ASCON-128 specifications. Functional correctness was verified against official NIST Known Answer Test vectors for both encryption and decryption, including rejection of corrupted tags.

Performance was evaluated through simulation at 100 MHz across six message sizes ranging from 16 to 400 bytes. The implementation achieves a message rate of 362,319 to 1,923,077 messages per second, exceeding the 4000 msg/s V2X requirement by factors of 90 to 423, depending on message size. Peak throughput reaches 1159 Mbps. All evaluated message sizes are complete within 2.76 µs, well within the defined constraints for V2X safety services [1].

Power was estimated using Vivado's post-implementation power analysis with 100% SAIF annotation coverage and High confidence on both the Artix-7 and Alveo U280 platforms. On-chip power is 0.173 W on Artix-7 and 3.226 W on the U280. The difference is largely due to device static power: 0.131 W versus 3.130 W, reflecting the U280's infrastructure overhead rather than the ASCON core itself. Logic power is 7 mW on both platforms, confirming that the cryptographic computation is energy-efficient and platform-independent.

The iterative architecture achieves low resource utilization, below 1000 LUTs and 620 flip-flops, while meeting the V2X workload requirements.

In summary, the results show that a single ASCON-128 hardware core at 100 MHz is well suited for V2X-oriented cryptographic acceleration, offering significant performance, low energy consumption, and compact resource utilization. These results support ASCON-128 as a viable building block for FPGA-based V2X

systems, pending end-to-end integration.

7.1 Future Work

Several paths exist for extending this work. First, a multi-core design in which multiple `ascon_cores` operate in parallel would increase the message rate proportionally, with minimal design complexity, which could assist in roadside unit deployments handling very high traffic densities beyond the baseline goal of 4000 msg/s set for this thesis.

Second, completing the implementation on the Alveo U280 with a full host interface, including PCIe and DMA transfers, would allow end-to-end system latency to be measured and would provide a more detailed picture of deployment readiness in a real V2X infrastructure scenario

Third, adding side-channel and fault-injection countermeasures, such as those studied by Kandi et al. [9], would meet the security requirements for deployment. Researching the power and area cost of these measures in the context of implementation is the next step.

Lastly, evaluating a pipelined variant design and measuring throughput against the resource overhead would provide a reference point for applications with more demanding throughput requirements than those of V2X.

Bibliography

[1] ETSI, "TS 122 185 V19.0.0: LTE; service requirements for V2X services," European Telecommunications Standards Institute, Technical Specification TS 122 185 V19.0.0, 2025. [Online]. Available: https://www.etsi.org/deliver/etsi_ts/122100_122199/122185/19.00.00_60/ts_122185v190000p.pdf.

[2] S. A. A. Hakeem, M. A. A. El-Gawad, and H. Kim, "Comparative experiments of v2x security protocol based on hash chain cryptography," *Sensors*, vol. 20, no. 19, p. 5719, 2020.

[3] H. Mosanaei-Boorani and S. Bayat-Sarmadi, "A digital signature architecture suitable for v2v applications," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 2, pp. 731–739, 2023.

[4] E. L. Cominetti, M. V. M. Silva, M. A. Simplicio Jr, H. K. Patil, and J. E. Ricardini, "Faster verification of v2x basic safety messages via message chaining," *Vehicular Communications*, vol. 44, p. 100 662, 2023.

[5] S. Vijayakumar, M. Mathivanan, M. Sathiya, and A. Kamaraj, "Optimised elliptic curve cryptography architecture for improved v2x communication," *Analog Integrated Circuits and Signal Processing*, vol. 123, no. 3, p. 52, 2025.

[6] A. Kandi, A. Baksi, T. Gerlich, S. Guilley, P. Gan, J. Breier, A. Chattopadhyay, R. R. Shrivastwa, Z. Martinasek, and S. Bhasin, "Hardware implementation of ascon," in *NIST Lightweight Cryptography Workshop*, vol. 2023, 2023.

[7] J. Kaur, A. C. Canto, M. M. Kermani, and R. Azarderakhsh, "A comprehensive survey on the implementations, attacks, and countermeasures of the current nist lightweight cryptography standard," *arXiv preprint arXiv:2304.06222*, 2023.

[8] M. Gladis Kurian and Y. Chen, "Ascon on fpga: Post-quantum safe authenticated encryption with replay protection for iot," *Electronics*, vol. 14, no. 13, p. 2668, 2025.

[9] A. Kandi, A. Baksi, P. Gan, S. Guilley, T. Gerlich, J. Breier, A. Chattopadhyay, R. R. Shrivastwa, Z. Martinásek, and S. Bhasin, "Side-channel and fault resistant ascon implementation: A detailed hardware evaluation," in *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, IEEE, 2024, pp. 307–312.

[10] A. R. Alharbi, A. Aljaedi, A. Aljuhni, M. K. Alghuson, H. Aldawood, and S. S. Jamal, "Evaluating ascon hardware on 7-series fpga devices," *Ieee Access*, vol. 12, pp. 149 076–149 089, 2024.

[11] M. Mirigaldi, V. Piscopo, M. Martina, and G. Masera, "The quest for efficient ascon implementations: A comprehensive review of implementation strategies and challenges," *Chips*, vol. 4, no. 2, p. 15, 2025.

[12] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, *Ascon – Lightweight Cryptography*, `https://ascon.isec.tugraz.at/index.html`, Accessed: 2026-06-07.

[13] K.-D. Nguyen, T.-K. Dang, B. Kieu-Do-Nguyen, D.-H. Le, C.-K. Pham, and T.-T. Hoang, "Asic implementation of ascon lightweight cryptography for iot applications," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 72, no. 1, pp. 278–282, 2024.

[14] A. Koppuravuri, H. Pasupuleti, S. Gvk, and J. Bapat, "A high throughput ascon architecture for secure edge iot devices," in *2024 37th International Conference on VLSI Design and 2024 23rd International Conference on Embedded Systems (VLSID)*, IEEE, 2024, pp. 486–491.

[15] K. Raj and S. Bodapati, "Fpga based light weight encryption of medical data for iomt devices using ascon cipher," in *2022 IEEE International Symposium on Smart Electronic Systems (iSES)*, IEEE, 2022, pp. 196–201.

[16] M. Ahmet, "High-performance fpga implementations of lightweight ascon-128 and ascon-128a with enhanced throughput-to-area efficiency," in *2024 17th International Conference on Information Security and Cryptology (ISC-Türkiye)*, IEEE, 2024, pp. 1–7.

Appendix A

Appendix

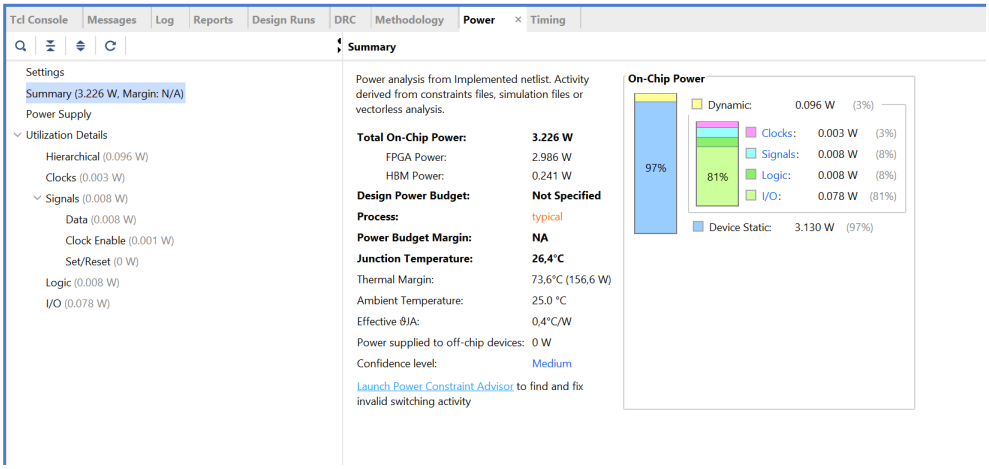


Figure A.1: Post-implementation power report for the ASCON design on the Alveo U280 platform.

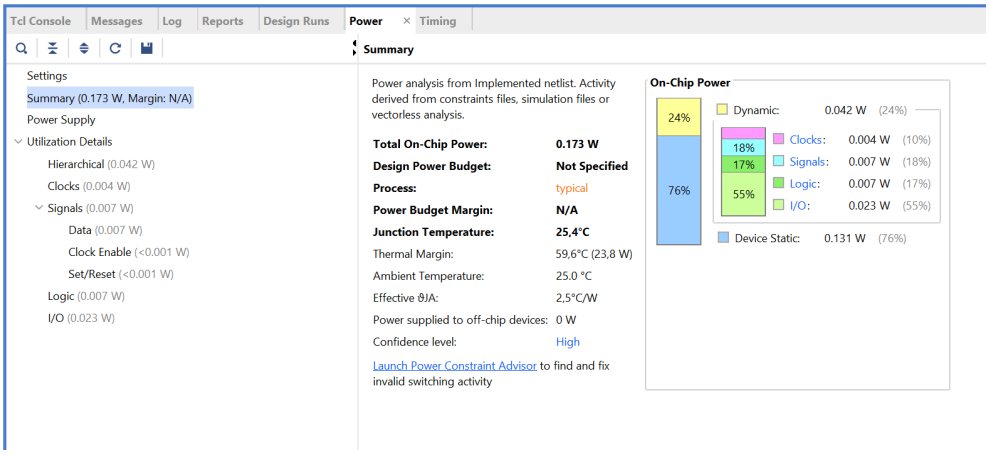


Figure A.2: Post-implementation power report for the ASCON design on the Artix-7 platform.

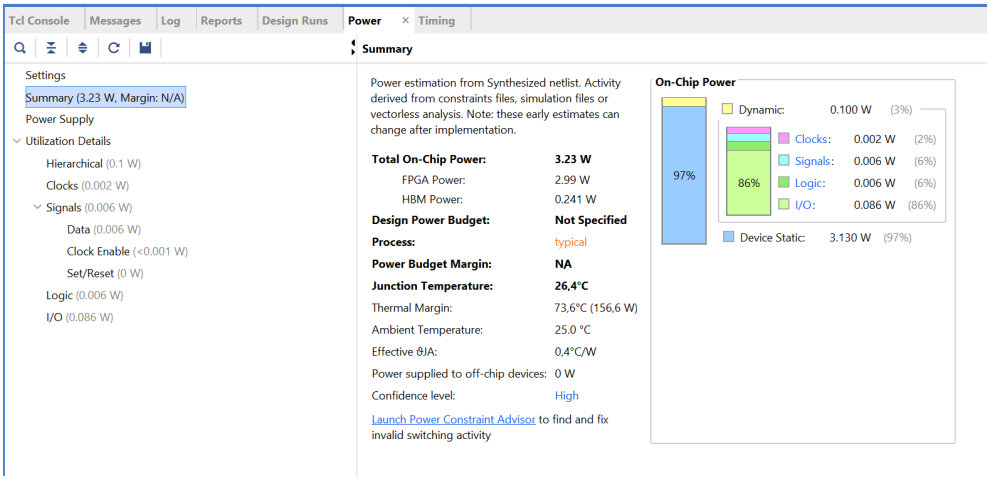


Figure A.3: Post-synthesis power report for the ASCON design on the Alveo U280 platform.

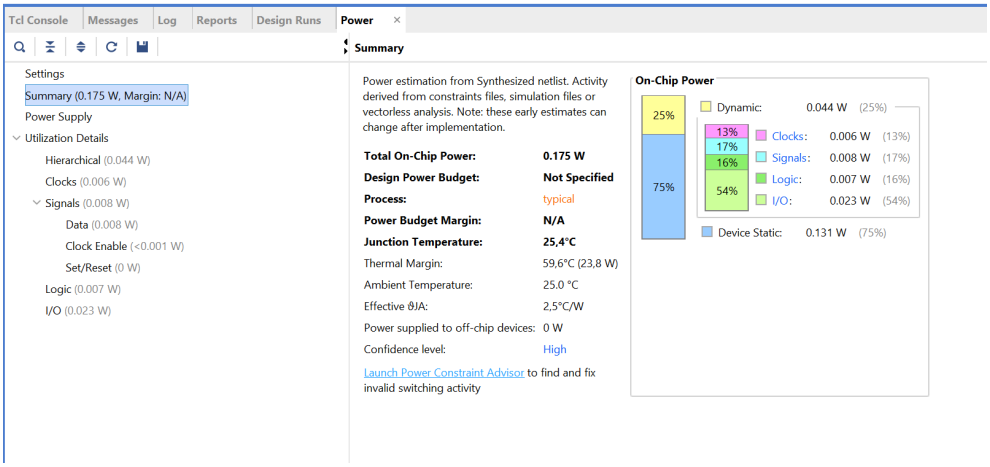


Figure A.4: Post-synthesis power report for the ASCON design on the Artix-7 platform.

```
------------------------------------------------------------
Msg Size (bytes) = 16
Cycles           = 59
Latency (us)     = 0.590000
Throughput Mbps  = 216.949153
Msg Rate msg/s   = 1694915.254237
Energy (uJ)      = 1.905700
INFO: [USF-XSim-96] XSim completed. Design snapshot 'ascon_throughput_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:04 ; elapsed = 00:00:08 . Memory (MB): peak = 6025.484 ; gain = 0.000
run all
------------------------------------------------------------
Msg Size (bytes) = 20
Cycles           = 60
Latency (us)     = 0.600000
Throughput Mbps  = 266.666667
Msg Rate msg/s   = 1666666.666667
Energy (uJ)      = 1.938000
------------------------------------------------------------
Msg Size (bytes) = 64
Cycles           = 87
Latency (us)     = 0.870000
Throughput Mbps  = 588.505747
Msg Rate msg/s   = 1149425.287356
Energy (uJ)      = 2.810100
------------------------------------------------------------
Msg Size (bytes) = 128
Cycles           = 123
Latency (us)     = 1.230000
Throughput Mbps  = 832.520325
Msg Rate msg/s   = 813008.130081
Energy (uJ)      = 3.972900
------------------------------------------------------------
Msg Size (bytes) = 320
Cycles           = 231
Latency (us)     = 2.310000
Throughput Mbps  = 1108.225108
Msg Rate msg/s   = 432900.432900
Energy (uJ)      = 7.461300
------------------------------------------------------------
Msg Size (bytes) = 400
Cycles           = 276
Latency (us)     = 2.760000
Throughput Mbps  = 1159.420290
Msg Rate msg/s   = 362318.840580
Energy (uJ)      = 8.914800
============================================================
PERFORMANCE TEST COMPLETE
============================================================
$finish called at time : 8485 ns : File "C:/Users/ander/VivadoASCON/ASCON/ASCON_power_artix/ASCON_power_artix.srcs/sim_1/new/ascon_throughput_tb.v" Line 180
```

Figure A.5: Throughput evaluation results for the ASCON implementation on the Alveo U280 platform.

```
Msg Size (bytes) = 16
Cycles           = 59
Latency (us)     = 0.590000
Throughput Mbps  = 216.949153
Msg Rate msg/s   = 1694915.254237
Energy (uJ)      = 0.102070
INFO: [USF-XSim-96] XSim completed. Design snapshot 'ascon_throughput_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:09 . Memory (MB): peak = 1555.535 ; gain = 0.000
run all
------------------------------------------------------------
Msg Size (bytes) = 20
Cycles           = 60
Latency (us)     = 0.600000
Throughput Mbps  = 266.666667
Msg Rate msg/s   = 1666666.666667
Energy (uJ)      = 0.103800
------------------------------------------------------------
Msg Size (bytes) = 64
Cycles           = 87
Latency (us)     = 0.870000
Throughput Mbps  = 588.505747
Msg Rate msg/s   = 1149425.287356
Energy (uJ)      = 0.150510
------------------------------------------------------------
Msg Size (bytes) = 128
Cycles           = 123
Latency (us)     = 1.230000
Throughput Mbps  = 832.520325
Msg Rate msg/s   = 813008.130081
Energy (uJ)      = 0.212790
------------------------------------------------------------
Msg Size (bytes) = 320
Cycles           = 231
Latency (us)     = 2.310000
Throughput Mbps  = 1108.225108
Msg Rate msg/s   = 432900.432900
Energy (uJ)      = 0.399630
------------------------------------------------------------
Msg Size (bytes) = 400
Cycles           = 276
Latency (us)     = 2.760000
Throughput Mbps  = 1159.420290
Msg Rate msg/s   = 362318.840580
Energy (uJ)      = 0.477480
```

Figure A.6: Throughput evaluation results for the ASCON implementation on the Artix-7 platform.